

Concurrent Task Dispatcher in Rust Report

For the project we implement a concurrent task dispatcher in rust. The main purpose of this assignment is to design and analyze a simplified operating system style scheduler that demonstrates concurrency topics. The system simulates a stream of incoming work, place that work into one or more queues, and assigns tasks to a bounded worker pool according to a scheduling policy. This project compares two scheduling strategies, FIFO scheduling and Optimized scheduling using separate CPU and IO queues. The workload also contains 1000 tasks with approximately 70% IO tasks and 30% CPU tasks.

This project is composed of several major components The first component would be the task generator, it automatically creates tasks using a fixed random seed, so results are reproducible. Each task contains task id, arrival time, durations, and task type CPU or IO. The difference between the two is that CPU tasks usually need longer execution times, while IO tasks are shorter and are quicker to compete.

As mentioned before, this project implements two scheduling systems. For the first implementation, it would be the FIFO scheduler. The FIFO scheduler uses a single shared queue meaning that tasks are inserted into the queue in arrival order and processed by worker threads using first in first out scheduling. Since this scheduler is simple and easy to analyze it acts as the baseline implementation. The second implementation is an optimized scheduler that separates CPU and IO tasks into different queues. Workers are assigned priorities based on task type. Most workers prioritize IO tasks while others prioritize CPU tasks. This approach improves concurrency and reduces contention between long CPU tasks and shorter IO tasks.

This project also uses several important data structures. Using VecDeque<task> a rust data structure is how queues were implemented because of the way it supports efficient insertion and removal operations from the front of the queue. Two major structs were used. The first, task truck stores task information including the task id, arrival time, duration, and task type. The second struct named Stats stores runtime metrics such as completed tasks, wait times, turnaround times, and worker utilization. An Enum named

TaskType is also used to distinguish between CPU tasks and IO tasks. These data structures helped organize the simulation while improving readability and maintainability.

Concurrency and Synchronization are also important parts of the project design, so this project uses rust concurrency primitives from standard library Shared state, Arc, Mutex, and Threads. Shared state includes the task queues and statistics structures because multiple worker threads access these resources simultaneously. Arc was used to provide shared ownership of queues and statistics across threads. Without Arc, multiple workers could not safely access the same shared data. Mutex protects shared queues and statistics from race conditions. Only one thread may modify shard data at a time. This prevents queue corruption and incorrect metric calculations. Rust threads are used for concurrent execution. Each worker processes tasks independently in parallel.

Two scheduling policies were also implemented for this project. For the first, the FIFO scheduler processes tasks in arrival order using one shared queue. This policy has its advantage being simple and predictable, which makes it easier to implement, however FIFO scheduling also introduces several problems. Long CPU tasks can block shorter IO tasks, increasing wait time and turnaround time. The optimized scheduling policy improves upon FIFO by separating CPU and IO tasks into independent queues. Its advantage is that worker specialization allows IO heavy workloads to complete more efficiently because shorter IO tasks are not delayed behind long CPU bound tasks. The scheduler also reduces queue contention because workers do not constantly compete for access to a single queue. Program test runs showed lower average wait times, lower turnaround times, and slightly lower make span values compared to FIFO scheduling. Based on the results the optimized scheduler performed better overall for the workload use of the project.

Although the optimized scheduler had its advantages of improved performance, it also introduced some disadvantages. The logic became more difficult to implement because workers needed fallback behavior when preferred queues were empty. It also introduced the possibility of unfairness, since most workers prioritize IO tasks, CPU tasks may occasionally experience higher wait times during heavy IO workloads. However, starvation is reduced because workers are still allowed to process tasks from the opposite queue if necessary.

In this project there were also some issues that happened during the development of the project. One of the issues was during the beginning of the project which was with the random number generation library used in rust. The issue was solved by updating the random generation methods. Another issue was the design, At first the optimized scheduler performed worse than the FIFO implementation the worker priorities and queue

organization were not balanced correctly. The issue was fixed by redesigning the worker assignment strategy and properly separating CPU and IO queues.

The program also collects several performance metrics during execution which include total tasks completed, makespan, average wait time, average turnaround time, maximum wait time, CPU usage, and worker utilization. Some additional metrics track CPU only wait time and IO only wait time. Because of these measurements it allowed for direct comparison between FIFO and optimized scheduling performance.

For this project, when getting the results from the output, it showed the differences between the two scheduling approaches. In the FIFO simulation, the average wait times and turnaround times were higher because all tasks shared a single queue, however in the optimized simulation, the average wait time and turnaround time improved because IO tasks could be processed more efficiently. In FIFO, long CPU tasks frequently delayed shorter IO tasks. The optimized simulation also achieved slightly higher CPU utilization and worker activity. These results show how scheduling policy and queue organization greatly affect overall system performance.

This project provided me with valuable experience with concurrency systems programming concepts. I would say that I learned the importance of synchronization when multiple threads share resources. I also learned that the scheduling policy has a direct impact on system efficiency and fairness. Overall, the concurrent task dispatcher successfully demonstrated multithreaded scheduling behavior using rust. This project implemented worker pools, queue-based scheduling, synchronization strategies, performance measurement, and multiple scheduling policies. It showed that the optimized scheduler performed better than the FIFO scheduler.